

WEBX Module 4

What is Python Flask?

Flask is a micro web framework for Python, designed to be lightweight and modular.

- Minimal core – provides routing, request/response handling, templating (Jinja2).
- Extensions – add databases, authentication, forms, etc.
- No built-in ORM – unlike Django, you choose your own.
- Built-in dev server and debugger.

Key use cases: REST APIs, small-to-medium web apps, prototypes.

Example – minimal app:

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, Flask!"

if __name__ == '__main__':
    app.run(debug=True)
```

Features of Python Flask

Feature	Description
Lightweight (micro)	No unnecessary components; you add only what you need.
Built-in dev server	<code>app.run()</code> with auto-reload, debugging.
Jinja2 templating	Dynamic HTML generation with inheritance, loops, filters.
URL routing	Decorators (<code>@app.route</code>) + variable rules (<code>/user/<name></code>).
Request/response	Global <code>request</code> and <code>make_response</code> objects.
Session & cookies	Client-side signed sessions; cookie methods.
Extensible	1000s of extensions (Flask-SQLAlchemy, Flask-Login, Flask-RESTful).

Feature	Description
HTTP methods support	GET, POST, PUT, DELETE, PATCH, etc. via <code>methods</code> parameter.
Modular blueprints	Organize large apps into reusable components.

Explain Flask Templates with an Example

Templates in Flask allow you to separate HTML structure from Python logic.

They use Jinja2 syntax: `{{ variable }}`, `{% control %}`, `{# comment #}`.

Folder structure:

```
app.py
templates/
    home.html
```

app.py:

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/user/<name>')
def user_profile(name):
    age = 25
    return render_template('home.html', username=name, age=age)
```

templates/home.html:

```
<!doctype html>
<html>
<body>
    <h1>Hello {{ username }}!</h1>
    <p>You are {{ age }} years old.</p>
    {% if age >= 18 %}
        <p>Adult</p>
    {% else %}
        <p>Minor</p>
    {% endif %}
</body>
</html>
```

Output: Visiting `/user/Alice` shows *"Hello Alice! You are 25 years old. Adult"*.

Explain the Concept of Flask Templates with an Example

Flask templates are dynamic HTML files that accept variables, loops, conditionals, and inheritance.

Folder structure:

```
app.py
templates/
    home.html
```

app.py:

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/user/<name>')
def user_profile(name):
    age = 25
    return render_template('home.html', username=name, age=age)
```

templates/home.html:

```
<!doctype html>
<html>
<body>
    <h1>Hello {{ username }}!</h1>
    <p>You are {{ age }} years old.</p>
    {% if age >= 18 %}
        <p>Adult</p>
    {% else %}
        <p>Minor</p>
    {% endif %}
</body>
</html>
```

Output: Visiting `/user/Alice` shows *"Hello Alice! You are 25 years old. Adult"*.

How to Set, Access, and Delete Cookies in Python Flask?

Use Flask's `make_response()` and `request.cookies` .

Set a cookie:

```
from flask import make_response

@app.route('/set')
def set_cookie():
    resp = make_response("Cookie set")
    resp.set_cookie('user', 'John', max_age=60*60*24) # 1 day
    return resp
```

Access a cookie:

```
from flask import request

@app.route('/get')
def get_cookie():
    user = request.cookies.get('user')
    return f"Cookie user = {user}"
```

Delete a cookie:

```
@app.route('/delete')
def delete_cookie():
    resp = make_response("Cookie deleted")
    resp.delete_cookie('user')
    return resp
```

Table of key parameters for `set_cookie` :

Parameter	Purpose
key	Cookie name
value	Cookie value
max_age	Lifetime in seconds (or use <code>expires</code>)
path	URL scope (default '/')
secure	Send only over HTTPS
httponly	Prevent JavaScript access

What is Flask URL Building?

URL building = generating URLs dynamically using Flask's `url_for()` function, instead of hardcoding strings.

Advantages:

1. Changes in route paths auto-propagate.
2. Handles variable rules (`/<id>`) automatically.
3. Works with blueprints.

Example:

```
from flask import Flask, url_for, redirect
app = Flask(__name__)

@app.route('/')
def home():
    return '<a href="' + url_for('profile', username='alice') + '">Profile</a>'

@app.route('/user/<username>')
def profile(username):
    return f"Hello {username}"
```

`url_for('profile', username='alice')` returns `/user/alice`.

Use cases:

- Redirects: `redirect(url_for('profile', username='bob'))`
- Static files: `url_for('static', filename='style.css')`
- Emails, navigation menus, API links.

HTTP Methods Provided by Python Flask

Flask supports all HTTP methods via the `methods` parameter in `@app.route`.

Method	Typical Use
GET	Retrieve data
POST	Submit new data

Method	Typical Use
PUT	Replace entire resource
PATCH	Partially update resource
DELETE	Remove resource
HEAD	Same as GET but no response body
OPTIONS	Describe allowed methods (CORS)

Example combining methods:

```
@app.route('/item/<int:id>', methods=['GET', 'PUT', 'DELETE'])
def handle_item(id):
    if request.method == 'GET':
        return f"Get item {id}"
    elif request.method == 'PUT':
        return f"Replace item {id}"
    elif request.method == 'DELETE':
        return f"Delete item {id}"
```

By default, only GET is allowed unless specified.

Explain REST API in Detail

REST API (Representational State Transfer API) is an architectural style for designing networked applications.

It uses HTTP as the protocol and treats every resource (e.g., user, product) as a URL.

Core constraints of REST:

1. Client–Server – separation of concerns.
2. Stateless – each request contains all needed info (no server-side sessions).
3. Cacheable – responses can be cached by client or intermediary.
4. Layered system – client cannot tell if talking directly to server or proxy.
5. Uniform interface – standard HTTP methods + resource identifiers.
6. Code-on-demand (optional) – server can send executable code.

Example CRUD operations for `/users` :

Action	HTTP Method	URL	Response
List users	GET	/users	200 OK + JSON list
Get one user	GET	/users/1	200 OK + user JSON
Create user	POST	/users	201 Created
Replace user	PUT	/users/1	200/204
Update user	PATCH	/users/1	200 OK
Delete user	DELETE	/users/1	204 No Content

Flask example:

```
from flask import Flask, request, jsonify

app = Flask(__name__)
users = {1: "Alice"}

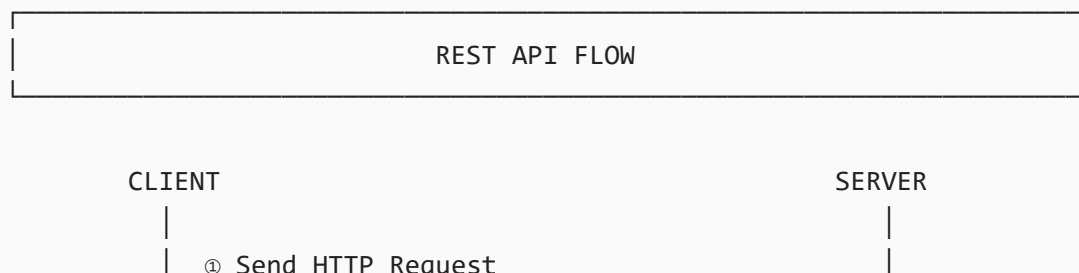
@app.route('/users/<int:uid>', methods=['GET'])
def get_user(uid):
    return jsonify({uid: users.get(uid)})

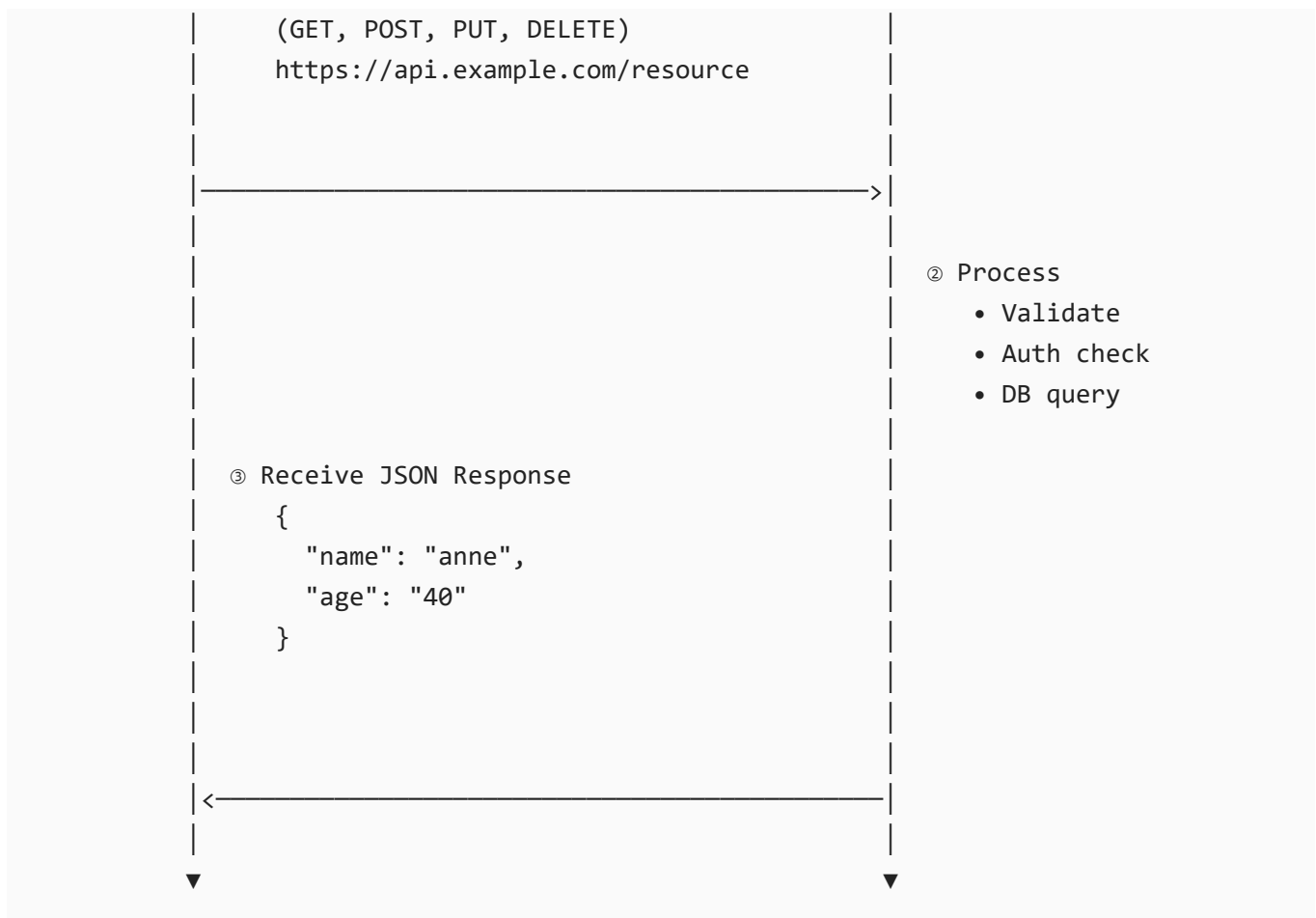
@app.route('/users/<int:uid>', methods=['DELETE'])
def del_user(uid):
    users.pop(uid, None)
    return '', 204
```

What Does REST Stand For? Explain REST API with a Basic Flow Diagram

REST stands for REpresentational State Transfer – introduced by Roy Fielding in 2000.

Basic REST API flow (text diagram):





Characteristic of REST in this diagram:

- Each request is stateless – contains all info (here, URL + method).
- Response contains representation (JSON) of resource state.
- Uniform interface via HTTP verbs.

Example cURL request to a REST API:

```
curl -X GET https://api.example.com/users/5 -H "Accept: application/json"
```